

Hamming Distance

The Hamming distance between two words of the same length is the number of positions where the letters differ.

$$H(x, y) = |\{i : x_i \neq y_i\}|$$

Question 1

Cost Measure 1: Steps

- Formula: $H(\text{node.state}, \text{goal})$ (Hamming Distance)
- Lower Bound: This formula represents the minimum cost of going from node state to goal state, assuming that each step correctly changes a new character in the node state.
- Overestimation: Because this formula correctly returns the minimum cost of going from node state to goal state, it can not overestimate the true optimal remaining cost.

Cost Measure 2: Scrabble

- Formula: $h_{\text{scrabble}}(n) = \sum_{i: n_i \neq g_i} \text{scrabble}(g_i)$ (sum of the Scrabble costs of goal letters at mismatched positions)
- Lower Bound: This formula represents the minimum cost of going from node state to goal state, assuming that each step correctly changes a new character in the node state.
- Overestimation: Because this formula correctly returns the minimum cost of going from node state to goal state, it can not overestimate the true optimal remaining cost.

Cost Measure 3: Frequency

- Formula: $H(\text{node.state}, \text{goal}) \cdot (1 + \text{Minimum Frequency Cost of Word in Dictionary with same length as goal state})$
- Lower Bound: This formula represents the minimum cost of making (Hamming Distance) changes to the node state to get to the goal state, it accounts for the unit cost for making a change, as well as the minimum frequency cost applied to each change.
- Overestimation: Because this cost is calculated using the minimum frequency cost of a word in our Dictionary, it is effectively the same heuristic as the steps measure, but safeguarded from overestimation.

Question 2

Word pair	Cost measure	UCS Cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
why -> ask	steps	5	773	5	16	97.93%
why -> ask	scrabble	9	765	9	13	98.30%
why -> ask	frequency	38.85540652	808	38.85540652	754	6.68%
bear -> duck	steps	4	691	4	6	99.13%
bear -> duck	scrabble	11	2981	11	6	99.80%
bear -> duck	frequency	38.2402284	1084	38.2402284	631	41.79%

1. Yes, UCS and A* always returned the same optimal cost.
2. For both word pairs, the scrabble cost method produced the largest percentage reduction in expanded nodes.
3. For both word pairs, the frequency cost measure led to the least percentage reduction in expanded nodes, indicating that the heuristic was the least informative out of the 3 heuristic functions.

4. While all of the heuristic functions generate a lower bound for the cost of going from node state to goal state, the steps and scrabble heuristics generated an inclusive lower bound, meaning a path with that exact cost did exist. The frequency heuristic used the minimum frequency cost for all words in the dictionary, meaning that unless path from node state to goal state only included words with the minimum value frequency, it often was an exclusive cost, that was not actually attainable, meaning that the heuristic always underestimated the true cost, leading to a smaller reduction in expanded nodes.

Question 3

Cost Measure 1: Steps

- $W1 \rightarrow W2 == W2 \rightarrow W1$: Yes
- Justification: The optimal cost between two words is simply the hamming distance, and hamming distance is commutative ($H(w_1, w_2) = H(w_2, w_1)$)
- Counterexample: N/a

Cost Measure 2: Scrabble

- $W1 \rightarrow W2 == W2 \rightarrow W1$: No
- Justification: The optimal cost between two words is based on hamming distance and scrabble costs of the replaced letters, hamming distance is commutative but scrabble cost is not.
- Counterexample: $\text{cat} \rightarrow \text{rat} = (1 + \text{scrabble}('r')) = 1 + 1 = 2$
 $\text{rat} \rightarrow \text{cat} = (1 + \text{scrabble}('c')) = 1 + 3 = 4$

Cost Measure 3: Frequency

- $W1 \rightarrow W2 == W2 \rightarrow W1$: No
- Justification: The optimal cost between two words is based on hamming distance and frequency costs of the replaced letters, hamming distance is commutative but frequency cost is not.
- Counterexample: $\text{cat} \rightarrow \text{rat} = 1 + \text{rarity}('r') = 1 + 8.398994 = 9.398994$
 $\text{rat} \rightarrow \text{cat} = 1 + \text{rarity}('c') = 1 + 6.886906 = 7.886906$

Question 4

- Model: GPT-5.3-Codex
- Prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed rarity-aware heuristic:

Let g be the goal word (length L), and let D_L be all legal dictionary words of length L .

For each position $i \in \{1, \dots, L\}$, compute:

$$\rho_i = \min\{\text{rarity}(w) : w \in D_L \text{ and } w[i] = g[i]\}$$

For a state n , let:

$$M(n) = \{i : n[i] \neq g[i]\}, \quad h_0(n) = |M(n)|$$

Define:

$$h_{\text{tar}}(n) = h_0(n) + \sum_{i \in M(n)} \rho_i$$

Why admissible:

- 1. Any legal action changes exactly one letter, so at least $h_0(n)$ actions are required.
- 2. For each mismatched position $i \in M(n)$, some step must set that position to $g[i]$.
- 3. At that step, the entered word must have letter $g[i]$ at position i , so its rarity is at least ρ_i .
- 4. Therefore every remaining path cost is at least $h_0(n) + \sum_{i \in M(n)} \rho_i$, so h_{tar} never overestimates.

Why more informative than h0:

- 1. $\rho_i \geq 0$ for all i , so $h_{\text{tar}}(n) \geq h_0(n)$ for every state n .
- 2. If at least one relevant $\rho_i > 0$, then for any state mismatching that position, $h_{\text{tar}}(n) > h_0(n)$, so it is strictly more informative for at least one state.

Test Results:

Word pair	Cost measure	UCS Cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
why -> ask	h_{tar}	38.85540652	808	38.85540652	510	36.88%
why -> ask	h_0	38.85540652	808	38.85540652	755	6.56%
bear -> duck	h_{tar}	38.2402284	1084	38.2402284	319	70.57%
bear -> duck	h_0	38.2402284	1084	38.2402284	720	33.58%

Based on the test results, this heuristic is admissible and useful. It improved the node expansion reduction by 30.32% and 36.99% in the why -> ask case and bear -> duck case, respectively. This makes sense, as it is a similar to my original heuristic, but instead of taking the minimum frequency of all words of length L, it only considers words of length L, and also have a specifc goal character at index i, for each correction needed in the node state.

Collaboration, sources, and generative AI policy Citation:

I used Github Copilot's as an auto-completion feature for some code that I generated during this assignment. The final code was all reviewed and approved by myself, and all reasoning/justification in this document was written by myself.

I also used ChatGPT to convert my text file writeup of this assignment to a pdf document for better readability and to match the professor's format requirements.